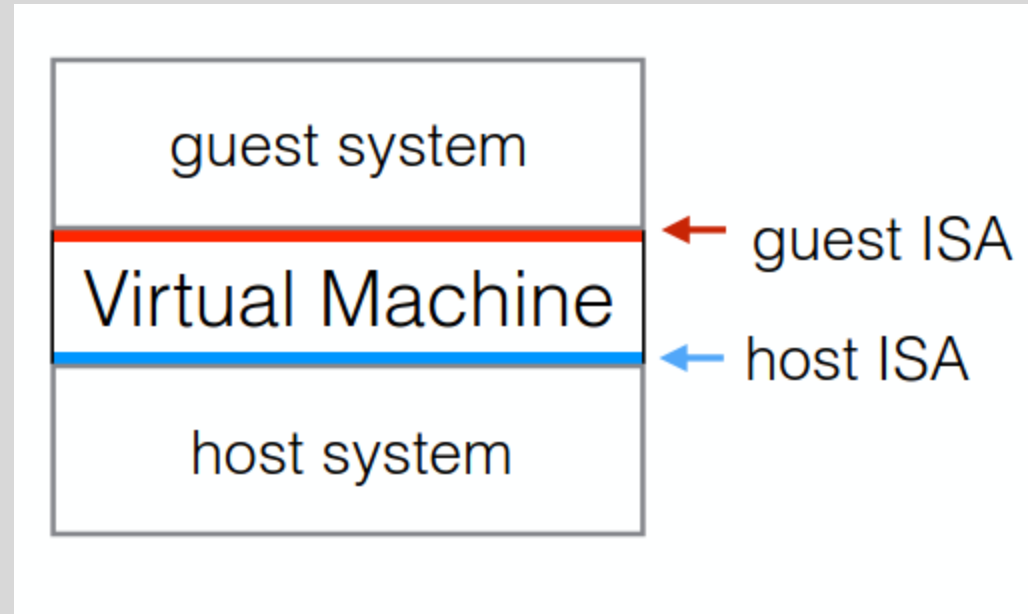




INTRODUCTION TO VM

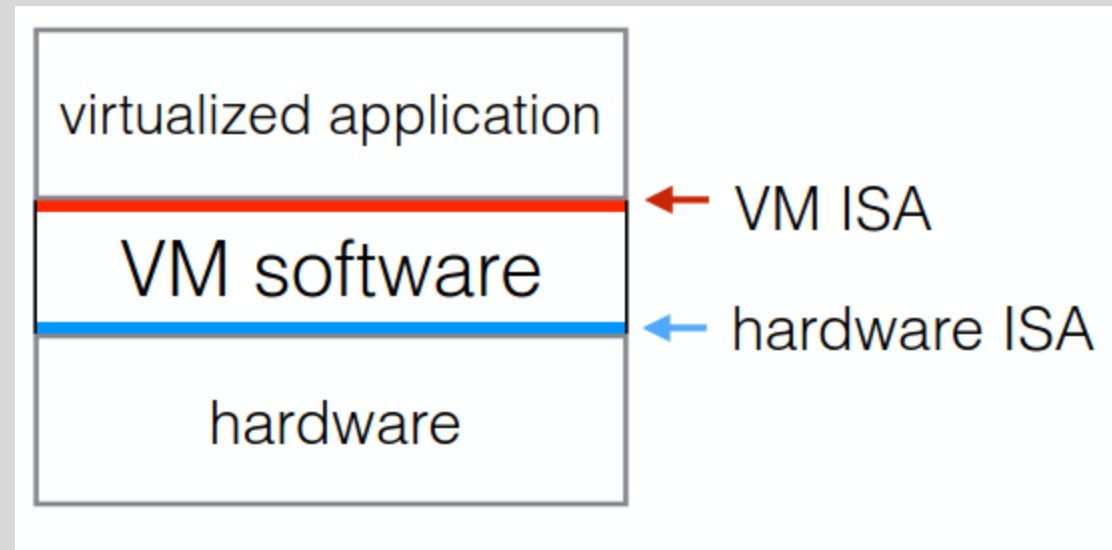
Generic VM architecture



What is a Virtual Machine?

- A software implementation of a machine architecture
 - Software needs hardware to run, so hardware is implied too
- Two machine architectures are involved: guest and host (although they might be the same!)
- The guest may be an abstract machine or an emulation of a real machine (i.e., also available as a hardware implementation)
- The host is usually hardware, but need not be (e.g., a language VM written in Java running on the JVM)

A typical VM



A smattering of VMs

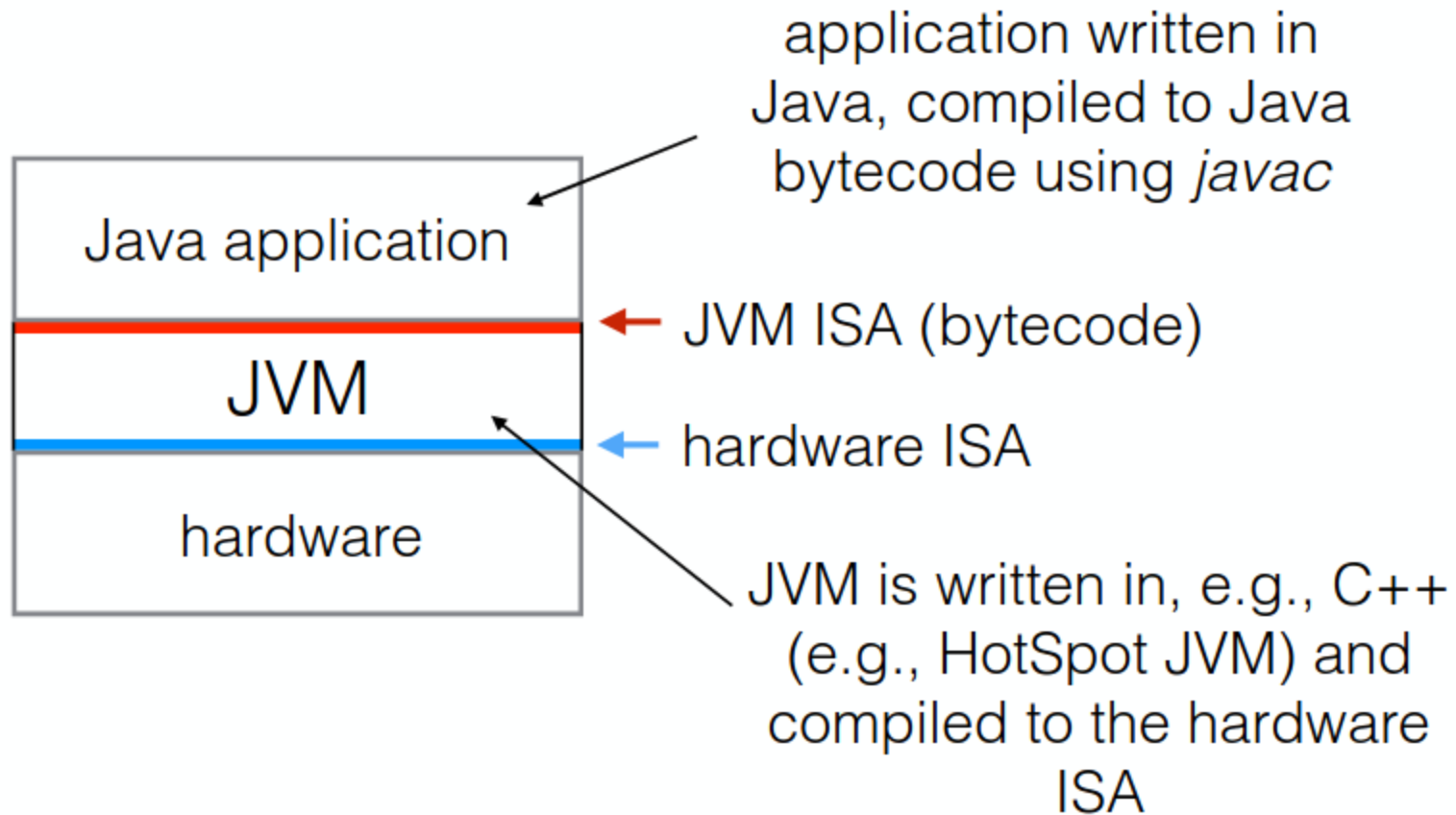
- **Java Virtual Machine (JVM):** a **language-level VM** designed to run Java bytecode.
- **VirtualBox:** a **hosted system VM** that allows multiple operating systems to run on a single physical machine.
- **Rosetta:** a **binary translation VM** that enables software compiled for one CPU architecture to run on another.
- **Dynamo:** a **dynamic binary optimization VM** that improves the performance of running programs by optimizing code on the fly.
- **Crusoe:** a **hardware/software co-designed VM**, where both hardware and software collaborate to emulate an x86-compatible processor.

Java Virtual Machine (JVM) – Example of a Language VM



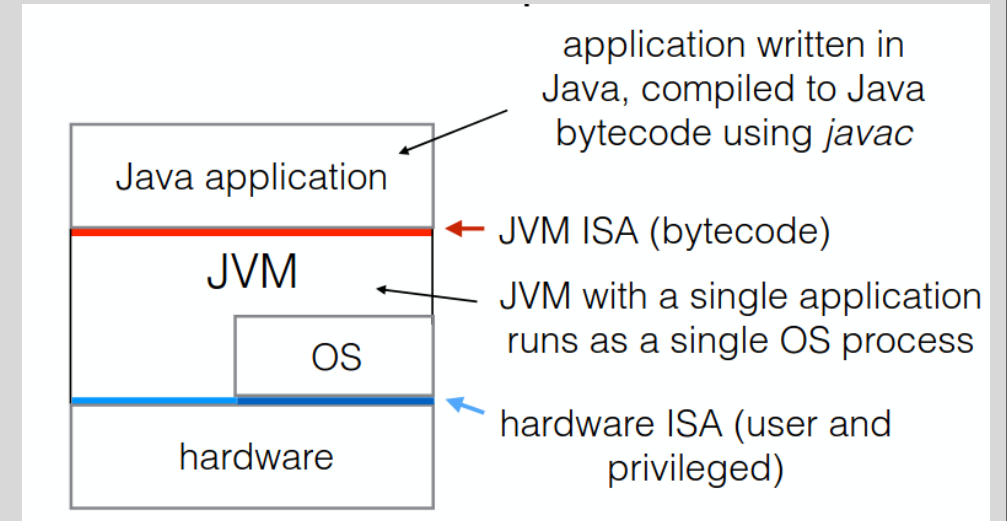
- **Designed for Java**, but now supports many other languages (e.g., Scala, Jython, Clojure, JRuby, JavaScript).
- Other languages also have their own VMs: for JavaScript, C#, Python, R, PHP, Ruby, Perl, Lua, etc.
- **Origin:** mid-1990s; inspired by earlier language VMs (like Pascal and Smalltalk).
- **Impact:** Java's popularity drove major advances in VM design: especially in **garbage collection** and **concurrency**.
- **Used everywhere:** from small devices (sensors, phones) to large systems (servers and clusters).

JVM



Widespread approach: JVM as a process

- **Application written in Java**
 - for example, `MyProgram.java`
- **Compiled to Java bytecode using `javac`**
 - When you run the Java compiler (`javac`), it converts your `.java` file into a `.class` file.
 - The `.class` file contains **Java bytecode**, this is **not machine code** yet, but an **intermediate form** understood by the JVM.
- **VM ISA (bytecode)**
 - This allows Java to be **platform-independent**, because the same bytecode can run on any system that has a JVM.
- **VM with a single application runs as a single OS process**
 - The **JVM itself** is just a **program** running on your operating system (Windows, Linux, macOS, etc.).
 - When you run a Java program, the OS sees it as **one process**, the JVM process, inside which your application runs.
 - So the JVM manages execution, memory, and threads on behalf of your Java program.
- **Hardware ISA (user and privileged)**
 - Beneath the JVM and OS, you have the **real hardware**, the CPU.
 - The CPU has its own **instruction set** (like x86, ARM, or RISC-V).
 - The operating system and JVM ultimately rely on the **hardware ISA** to execute the actual machine instructions.



Aside: JVM concurrency

- **Java** was one of the **first popular programming languages** to have **concurrency built in** from the start (in the mid-1990s).
 - That means it had built-in keywords like `Thread`, `synchronized`, and `wait/notify`
- **Early JVMs (Green Threads)**
 - The **first versions of the JVM** did not use the OS threads.
 - Instead, the JVM **simulated multithreading by itself**, using only **one real OS thread**.
 - It **manually switched between Java threads** inside the JVM, kind of like juggling tasks one at a time.
 - This switching was done **in software**, usually **between bytecode instructions** (so, after one piece of work finished, it could switch to another thread).
 - This approach was called **“Green Threads.”**

_cont- Why “Green Threads”?

- The term came from the **Green Project**, which was the original Sun Microsystems project that created Java (then called *Oak*).
- So, “Green Threads” are named after that project.
- Technically, this approach is also known as “**cooperative multithreading.**”
 - *Cooperative* means each thread *voluntarily* gives up control, rather than being preempted by the OS.

Modern JVMs

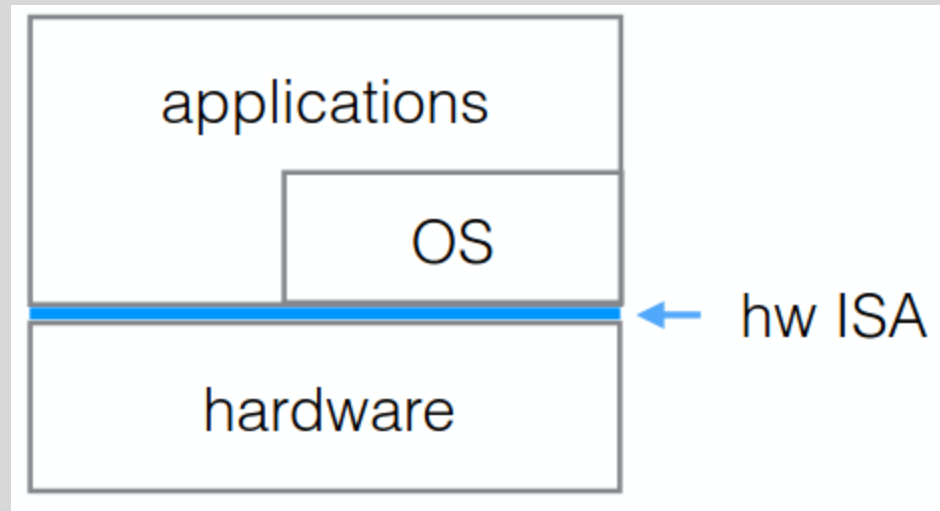
- Modern Java Virtual Machines no longer do this manually.
- Instead, they **use the operating system's native threads**.
- Each **Java thread** corresponds to an **OS thread**, and the OS handles scheduling automatically.
- This allows Java programs to take advantage of **multiple CPU cores** and real **parallel execution**.

Example #2: VirtualBox

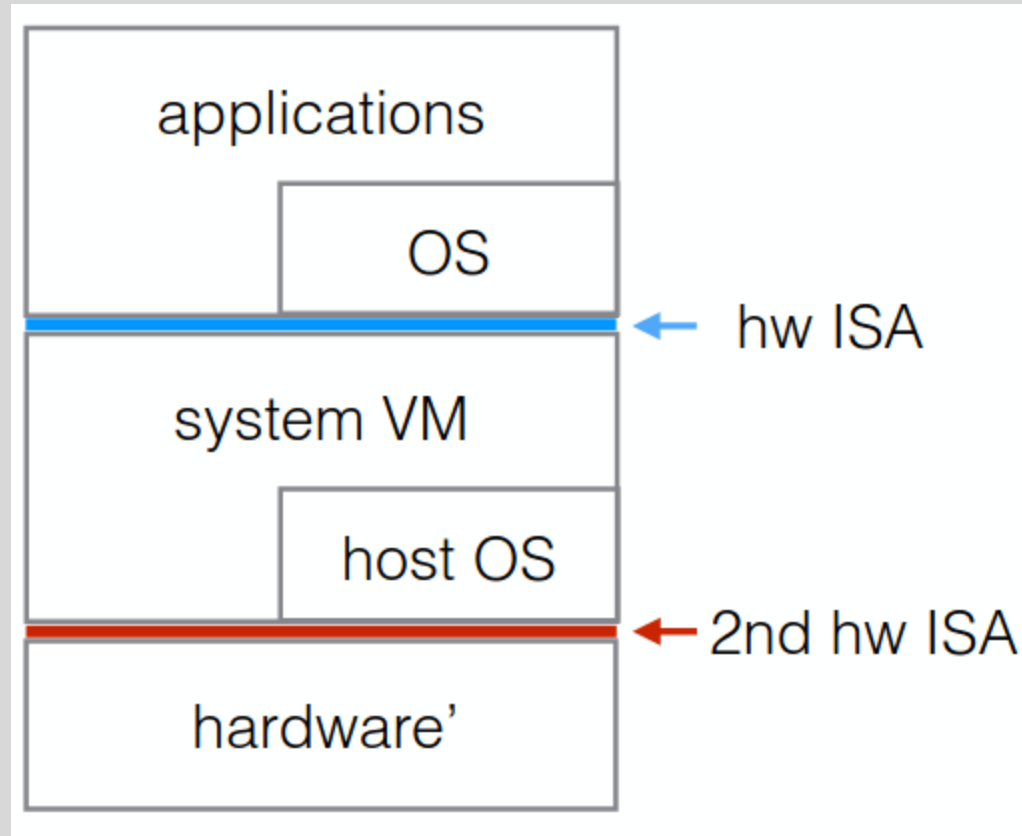
- Runs x86 systems (OS and applications) on x86. Why?
 - To run a different OS under the host OS
 - Example: Running Ubuntu inside Windows.
 - To run multiple guest systems simultaneously
 - Example: Developers testing apps on different OS versions simultaneously.
- Type of VM
 - **System VM** → runs a full OS and its applications.
 - **Hosted VM** → runs *on top* of another OS (the host OS provides drivers, memory, I/O, etc.).
- Many other similar products: VMware (various), QEMU, ...



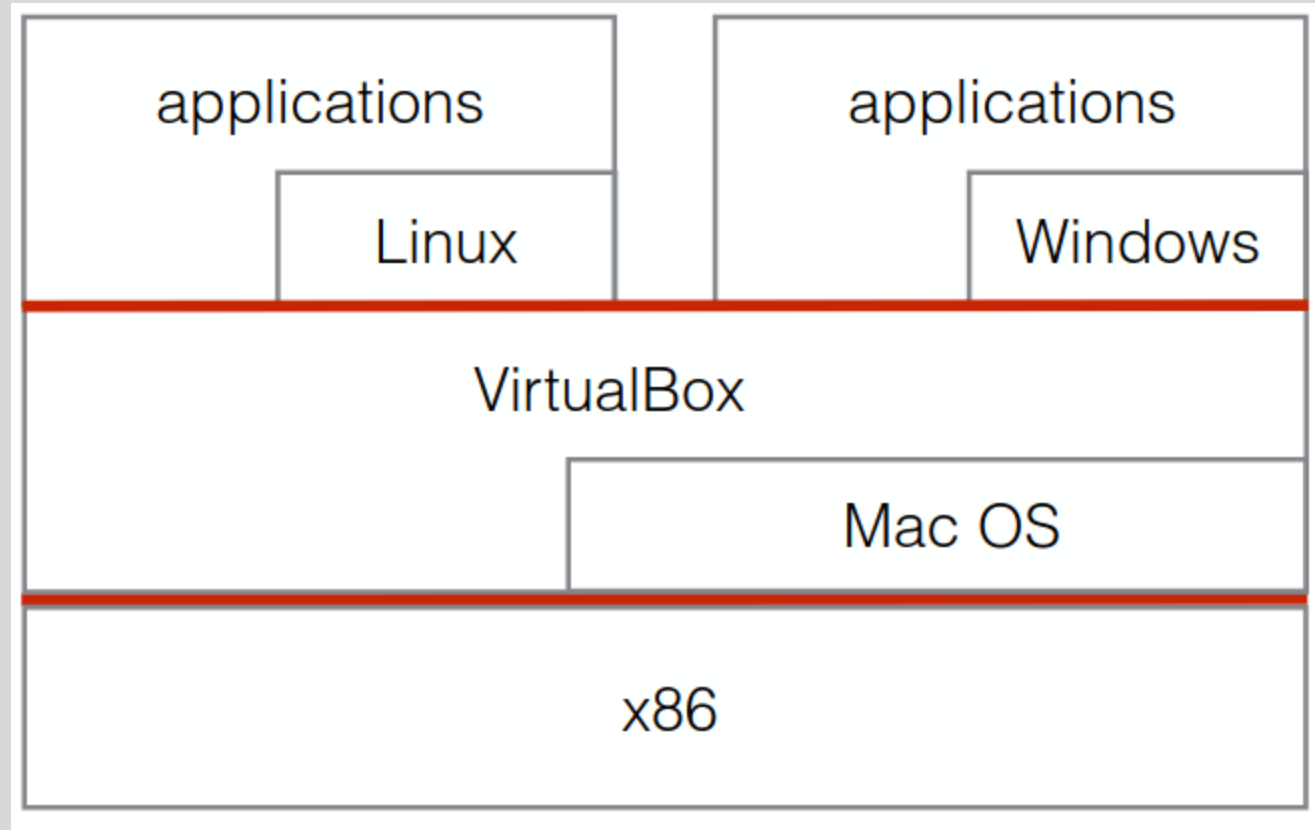
Typically we have



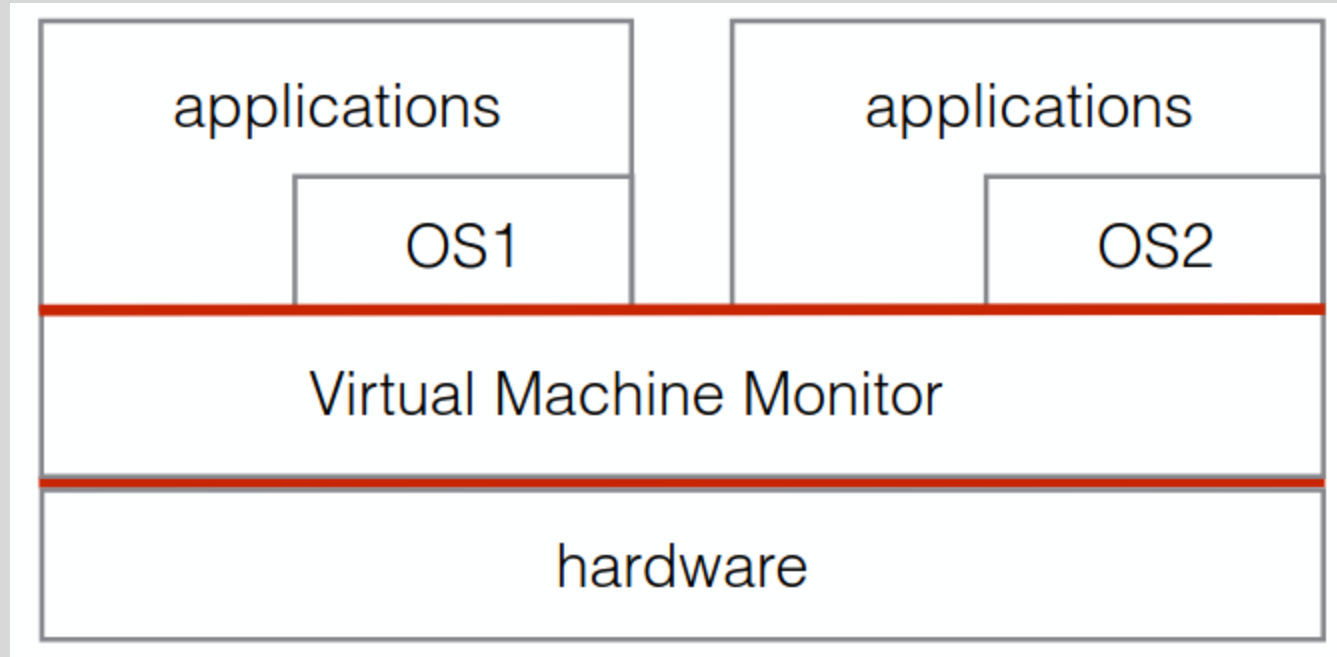
Architecture of Hosted System VMs



VirtualBox example



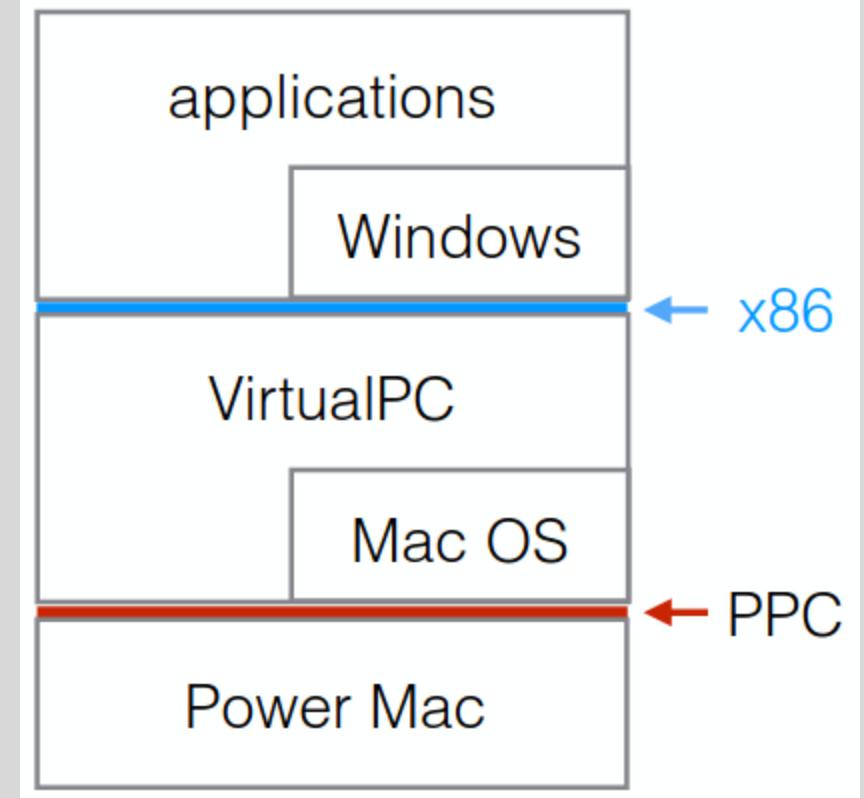
“Classic” System VM architecture (e.g., IBM System/370)



ISAs are the same, but there is no host OS

A Whole-System VM: Virtual PC (1997)

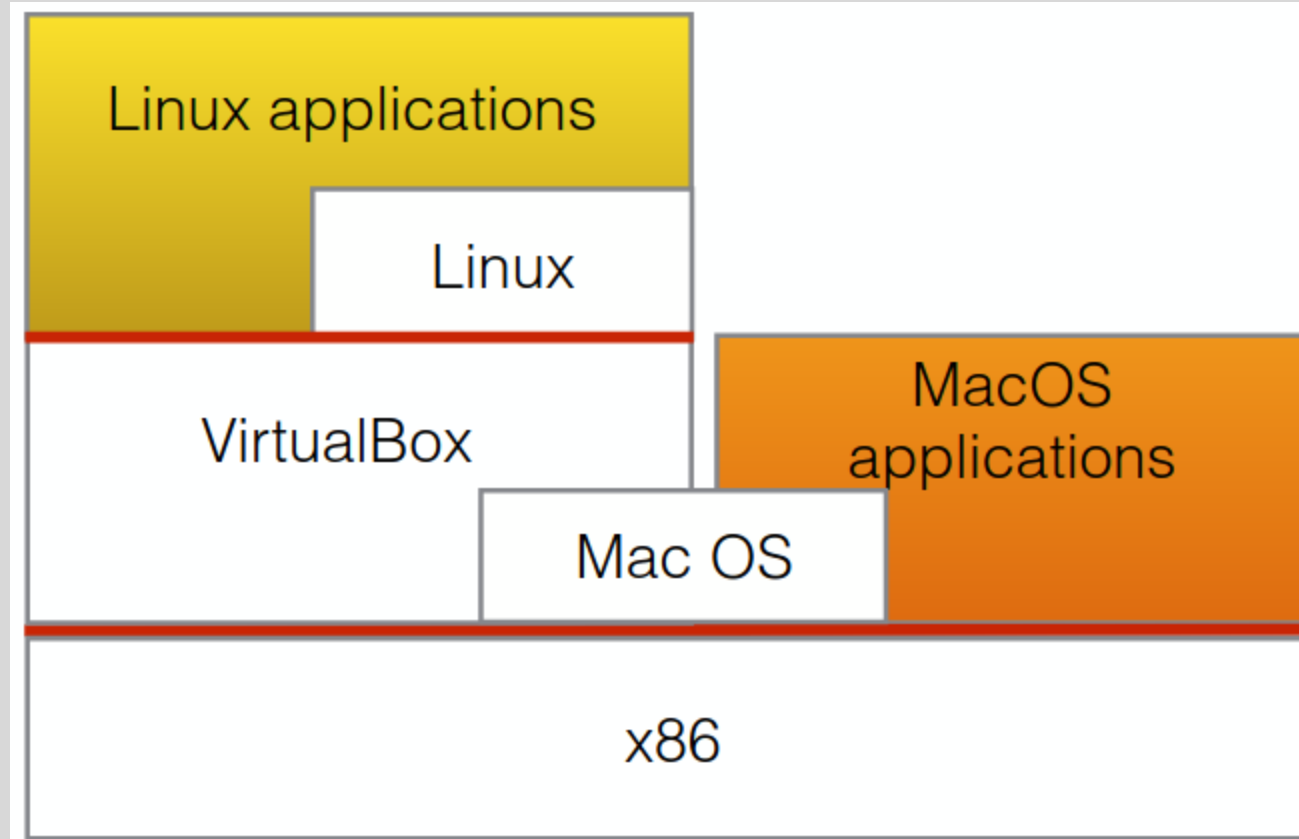
- **Power Mac (hardware): PPC architecture**
 - The physical machine is a **Power Mac** that uses a **PowerPC (PPC) CPU architecture**.
 - This is a different CPU design from x86.
- **Mac OS running on Power Mac**
 - The **host operating system** running directly on the Power Mac hardware is **Mac OS**, designed for PPC.
- **VirtualPC (VM software)**
 - **VirtualPC** is a **virtual machine** running **on top of Mac OS**.
 - It **emulates an x86 machine** inside the PPC Mac environment.
 - This means it translates or simulates x86 instructions so they can run on PPC hardware.
- **Windows running inside VirtualPC (guest OS)**
 - Inside the VirtualPC VM, you have **Windows** running as a guest operating system.
 - Windows expects an x86 CPU, so VirtualPC provides that environment virtually.
- **Applications running on Windows**
 - Finally, applications designed for Windows run inside this guest Windows OS.
 - They believe they are running on a real x86 PC, but actually, VirtualPC makes it happen on a Power Mac.



Definitions

- A **Process VM** implements an **ABI** (Application Binary Interface: the combination of a user-level ISA and an OS system call interface)
- A **System VM** implements both the hardware user and system ISA
- A Hosted System VM has a host OS
- A Classic System VM has a Virtual Machine Monitor (not a full OS; you can't run applications on the VMM as it does not implement the ABI)

Hosted VM- two levels for applications



Apple's Rosetta (2005)

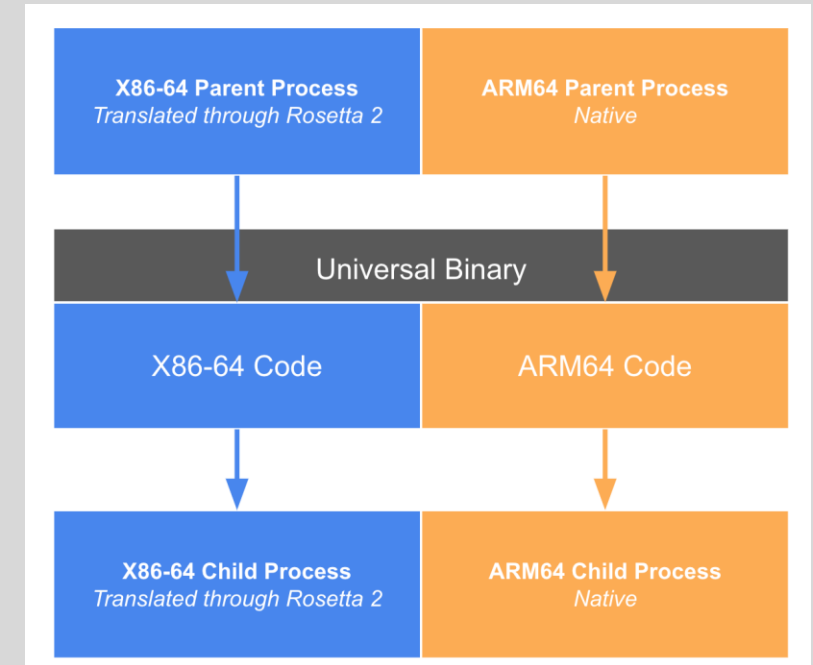
- In 2005, Apple transitioned from **PowerPC (PPC)** processors to **Intel x86** processors for their Macs.
- **Problem:** Existing Mac applications were built for PPC and would not run natively on the new Intel Macs.
- **Solution:** Apple introduced **Rosetta**, a **binary translator** that emulated the PPC architecture on Intel hardware.
- This allowed users to **run older PPC apps on the new Intel Macs** seamlessly while developers worked on porting their software to Intel.
- Rosetta was based on **QuickTransit**, a technology from Transitive Technologies (later acquired by IBM in 2009).

Current trend- Has Apple moved back?

- Yes, in 2020, Apple transitioned again but to ARM, this time from Intel x86 chips back to its own Apple Silicon (ARM-based) processors (like the M1 and M2).
- To support older Intel apps on the new ARM-based Macs, Apple introduced Rosetta 2, a new binary translator that translates x86 instructions to run on Apple Silicon.
- This move shows Apple's focus on custom ARM chips for better performance and efficiency, while still supporting legacy software via Rosetta 2.

Rosetta 2

- Apple bundles both Intel (x86-64) and ARM64 code into one **Universal Binary**.
- On Apple Silicon Macs, **ARM64 code runs directly (native)**, while **x86-64 code runs through Rosetta 2 translation**.
- Rosetta 2 dynamically translates x86-64 instructions to ARM64, enabling legacy Intel apps to run seamlessly on new ARM Macs.
- This process is like a lightweight VM focused on CPU translation rather than full system virtualization.



Emulator

- A machine emulator typically mimics a real instruction set; sometimes just user mode
- Not usually anywhere near as fast as the original system
- Sometimes called a simulator; can omit detail (e.g., side effects, or a precise accounting of time), allow extra observation, attached microarchitecture models, simulated devices
- Examples: QEMU; Simics (simulator)
- Aside: Emulation is not limited to machines: e.g., CodeWeavers' Crossover emulates the Windows API on a Mac; an in-circuit emulator mimics the electrical interface



Thank You